Bit Manipulation

How Computers "See" Numbers (Base 2)

We count in Base 10 (Decimal) because we have 10 fingers.
Each position in a number represents a power of 10 (1s, 10s, 100s).
Computers use Base 2 (Binary) because they operate on electrical switches: 1 is ON, 0 is OFF.

Each position in a binary number represents a power of 2, reading from right to left, starting at 2^0.

Let's look at the decimal number 13:

13 is made of 8 + 4 + 0 + 1.

In powers of 2: (1 * 2^3) + (1 * 2^2) + (0 * 2^1) + (1 * 2^0).

In binary: 1101.

In standard CP environments, a standard int is 32 bits long.
So, 13 isn't just 1101, to the compiler, it actually looks like 28 zeros followed by 1101:

00000000000000000000000000001101

Always remember powers of 2:

```
2^0  = 1
2^1  = 2
2^2  = 4
2^3  = 8
2^4  = 16
2^5  = 32
2^6  = 64
2^7  = 128
2^8  = 256
2^9  = 512
2^10 = 1024
```

Binary means:

each bit n whether that power of 2 is used or not

So binary is basically:

number = sum of powers of 2

Numbers Between Powers Are Just Additions

Every number is:

power_of_two + remainder

Example: 19

Nearest power: 16

Remaining: 3

Now use known binary:

16 = 10000
3 = 00011

Combine:

10011

Example 45

45 = 32

Break:

45 = 32 + 8 + 4 + 1

Now instantly:

32 → 1____
8 → _1_
4 → __1_
1 → ___1

Answer:

101101

Numbers after powers of two follow a pattern.

Example after 8:

8 = 1000
9 = 1001
10 = 1010
11 = 1011
12 = 1100
13 = 1101
14 = 1110
15 = 1111

So once you know 8, you instantly know 9–15.

Memorize this block:

```
 1  1
 2  10
 3  11
 4  100
 5  101
 6  110
 7  111
 8  1000
 9  1001
10  1010
11  1011
12  1100
13  1101
14  1110
15  1111
```

The Sign Bit & Two's Complement

Before letting the operators, you need to know how negative numbers are handled.
The leftmost bit (the 31st bit in a 32-bit int, zero-indexed) is the sign bit.

- If it's 0, the number is positive.
- If it's 1, the number is negative.

Computers use a system called Two's Complement to represent negative numbers.
To get the negative version of a number:

1. Flip all the bits (0s become 1s, 1s become 0s).
2. Add 1.

Why this matters for CP:
This explains why the maximum positive 32-bit integer is 2147483647 (which is $2^{31} - 1$, since one bit is reserved for the sign).
If you add 1 to it, the binary overflows into the sign bit, instantly turning your number into -2147483648.

The Big Three Operators: AND, OR, XOR

These operations compare two numbers bit by bit, matching up their respective positions.

Bitwise AND (&)

The AND operator returns 1 only if both corresponding bits are 1.

```
Bit A    Bit B    A & B

0        0        0
0        1        0
1        0        0
1        1        1
```

Mental Model:

Think of AND as an intersection or a filter. It requires strict agreement.

Example: 13 & 6

- 13 in binary: 1 1 0 1
- 6 in binary: 0 1 1 0

Result: 0 1 0 0 (which is 4 in decimal).

Bitwise OR (|)

The OR operator returns 1 if at least one of the corresponding bits is 1.

```
Bit A    Bit B    A | B

0        0        0
0        1        1
1        0        1
1        1        1
```

Mental Model:

Think of OR as a union or an accumulator. It preserves every 1 it sees.

Example: 13 | 6

13: 1 1 0 1
6: 0 1 1 0

Result: 1 1 1 1 (which is 15 in decimal).

Bitwise XOR (^)

The XOR (Exclusive OR) operator returns 1 if the bits are different, and 0 if they are the same.

```
Bit A    Bit B    A ^ B

0        0        0
0        1        1
1        0        1
1        1        0
```

Mental Model:

XOR is arguably the most tested bitwise operator in CP. You should memorize two ways to think about it:

1. The Difference Detector: It flags exactly where two numbers disagree.
2. Addition without Carry: If you add two bits in binary, 1 + 1 = 10 (which is 0, carry the 1). XOR does the exact same thing but throws away the carry. 1 ^ 1 = 0.

Example: 13 ^ 6

13: 1 1 0 1
6: 0 1 1 0

Result: 1 0 1 1 (which is 11 in decimal).

The NOT Operator (~)

Unlike the previous three, NOT is a unary operator—it only takes one number.
It flips every single bit.

```
Bit A        ~A

0            1
1            0
```

Mental Model:

It's the ultimate inverter.
Because of Two's Complement (mentioned above), there is a strict mathematical relationship you can use: ~x = -x - 1.

If you ~13 in a 32-bit integer, you don't just flip the 1101; you also flip the 28 leading zeros into ones, which results in -14.

Bit Shifting (<< and >>)

Bit shifting literally slides the binary representation of a number to the left or right, adding zeros to fill the empty spaces.

Left Shift (<<)

x << y shifts the bits of x to the left by y positions.

Example: 3 << 2

3 in binary: ...00000011

Shift left by 2: ...00001100 (which is 12).

The Math Trick: Shifting left by y is equivalent to multiplying by 2^y. 3 * (2^2) = 12.

Right Shift (>>)

x >> y shifts the bits of x to the right by y positions.
The bits that fall off the right edge are deleted.

Example: 13 >> 1

13 in binary: ...00001101

Shift right by 1: ...00000110 (which is 6).

The Math Trick: Shifting right by y is equivalent to integer division by 2^y. 13 / (2^1) = 6.

If you want to create a bitmask by shifting 1 to the 40th position, writing 1 << 40 will cause Undefined Behavior and result in a wrong answer penalty. In C++, 1 is a 32-bit integer by default, and you cannot shift a 32-bit integer by 40. You must tell the compiler to treat 1 as a 64-bit integer by writing 1LL << 40.